Training Robust Neural Networks Using Lipschitz Bounds

Patricia Pauli, Anne Koch, *Graduate Student Member, IEEE*, Julian Berberich, *Graduate Student Member, IEEE*, Paul Kohler, and Frank Allgöwer, *Member, IEEE*

***Abstract*—Due to their susceptibility to adversarial perturbations, neural networks (NNs) are hardly used in safety-critical applications. One measure of robustness to such perturbations in the input is the Lipschitz constant of the input-output map defined by an NN. In this letter, we propose a framework to train multi-layer NNs while at the same time encouraging robustness by keeping their Lipschitz constant small, thus addressing the robustness issue. More specifically, we design an optimization scheme based on the Alternating Direction Method of Multipliers that minimizes not only the training loss of an NN but also its Lipschitz constant resulting in a semidefinite programming based training procedure that promotes robustness. We design two versions of this training procedure. The first one includes a regularizer that penalizes an accurate upper bound on the Lipschitz constant. The second one allows to enforce a desired Lipschitz bound on the NN at all times during training. Finally, we provide two examples to show that the proposed framework successfully increases the robustness of NNs.**

***Index Terms*—Linear matrix inequalities, neural networks, robustness.**

I. Introduction

NEURAL networks (NNs) and deep learning have lately been successful in many fields [1], [2], where they are mostly used for classification and segmentation problems, as well as in reinforcement learning [3]. The main advantages of NNs are that they can be trained straightforwardly using backpropagation and as universal function approximators they have the capability to represent complex nonlinearities. While NNs are powerful and broadly applicable they lack rigorous guarantees which is why they are typically not applied to safety-critical applications such as medical devices and autonomous driving. Adversarial attacks can easily deceive an NN by adding imperceptible perturbations to the input [4] which is a problem that has recently been tackled increasingly in a number of different ways such as adversarial training [5] and defensive distillation [6], neither of which providing formal guarantees. However, imposing during training that an NN is provably robust against norm-bounded adversarial perturbations, yields robustness certificates [7], [8], e.g., by maximizing margins [9], as well as using Lipschitz constants as a robustness measure that indicate the sensitivity of the output to perturbations in the input. The problem of estimating and providing guaranteed upper bounds on the global and local Lipschitz constant of NNs has been studied heavily [10]–[15], where some approaches exploit explicitly the structure of ReLU activation functions [14], [15]. Naturally, there is a trade-off between accuracy and complexity. Based on an accurate Lipschitz constant estimation for general activation functions [13], we propose a framework for training of robust NNs that encourages a small Lipschitz constant or respectively, a guaranteed upper bound on the NN's Lipschitz constant.

Trivial Lipschitz bounds of NNs can be determined by the product of the spectral norms of the weights [4] which is used during training in [16], [17]. Similar to [18], we use the Lipschitz constant as a regularization functional. However, they use local Lipschitz constants whereas we penalize the global one. In [13], Fazlyab *et al.* propose an interesting new estimation scheme for more accurate upper bounds on the Lipschitz constant than the weights' spectral norms exploiting the structure of the nonlinear activation functions. Activation functions are monotonically increasing functions with bounded slopes, which is used in [13] to state the property of *slope-restriction* as an incremental quadratic constraint and then formulate a semidefinite program (SDP) that determines an upper bound on the Lipschitz constant. In [13], three variants of the Lipschitz constant estimation framework are proposed trading off accuracy and computational tractability. In this letter, we disprove by counterexample the most accurate approach presented in [13], and we employ the other approaches for training of robust NNs. More specifically, we include the SDP-based Lipschitz bound characterization of [13] in the training procedure via an Alternating Direction Method of Multipliers

Manuscript received September 14, 2020; revised November 27, 2020; accepted December 23, 2020. Date of publication January 11, 2021; date of current version June 23, 2021. This work was supported by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) through Germany's Excellence Strategy—EXC under Grant 2075 - 390740016. The work of Patricia Pauli, Anne Koch, and Julian Berberich was supported by International Max Planck Research School for Intelligent Systems (IMPRS-IS). Recommended by Senior Editor C. Seatzu. *(Corresponding author: Patricia Pauli.)*

The authors are with the Institute for Systems Theory and Automatic Control, University of Stuttgart, 70550 Stuttgart, Germany (e-mail: patricia.pauli@ist.uni-stuttgart.de; anne.koch@ist.uni-stuttgart.de; julian.berberich@ist.uni-stuttgart.de; paulkohler@posteo.de; frank.allgower@ist.uni-stuttgart.de).

Digital Object Identifier 10.1109/LCSYS.2021.3050444

2475-1456 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on September 29,2025 at 07:58:14 UTC from IEEE Xplore. Restrictions apply.

(ADMM) scheme. We present two versions of the training method, (i) a regularizer rendering the Lipschitz constant small and (ii) enforces guaranteed upper bounds on the Lipschitz constant during training.

The main contributions of this letter are the two training procedures that provide robust multi-layer NNs in the notion of Lipschitz continuity, i.e., with guaranteed and respectively, minimum Lipschitz bounds. To the best of our knowledge this is the first training method that is based on the tight Lipschitz constant estimation proposed in [13] for which we also show that it requires a modification for the least conservative choice of decision variables. This letter is organized as follows. In Section II, we introduce the Lipschitz constant estimation for NNs based on [13] but disprove their most accurate Lipschitz estimator. In Section III, we present a training procedure with Lipschitz regularization and outline the setup of the optimization problem that is solved using ADMM. Subsequently, we introduce a variation of the proposed procedure that allows to enforce Lipschitz bounds on the NN, yielding an NN with guaranteed Lipschitz bounds, and finally, we discuss the convergence and the computational tractability of the ADMM scheme. In Section IV, we provide two examples on which we successfully apply the proposed methods.

II. Lipschitz Constant Estimation

In this section, we briefly introduce robustness in the context of NNs. We then state a method to estimate the Lipschitz constant of an NN based on [13] and finally, we argue why one of the methods for Lipschitz constant estimation proposed in [13] is incorrect.

A. Robustness of NNs

A robust NN should not change its prediction if the input is perturbed imperceptibly and to quantify robustness, a suitable robustness measure has to be defined. One definition is that perturbations from a norm-bounded uncertainty set may not alter the prediction. Alternatively, using probabilistic approaches, random perturbations do not change the prediction with a certain probability. While the aforementioned robustness measures are restricted to classification problems and as they only provide guarantees in sets around training data points, we choose a more general alternative without these disadvantages to quantify robustness, Lipschitz continuity. A function $f : \mathbb{R}^n \to \mathbb{R}^m$ is globally Lipschitz continuous if there exists an $L \geq 0$ such that

$$\|f(x) - f(y)\| \leq L\|x - y\| \quad \forall x, y \in \mathbb{R}^n. \tag{1}$$

The smallest L for which (1) holds is the Lipschitz constant L^*. If the input changes from x to y, the Lipschitz constant gives an upper bound on how much the output f changes. Hence, a low Lipschitz constant indicates low sensitivity which is equivalent to high robustness. In this letter, we aim to minimize the Lipschitz constant or respectively bound the Lipschitz constant from above during training to increase the robustness of the resulting NN.

Regularization, i.e., adding a penalty term to the objective function of the NN, is a prevalent measure in NN training in order to prevent overfitting. Penalizing the weights, e.g., with L1 or L2 regularization, counteracts the fit of sudden peaks and outliers, promotes better generalization, and smoothens the resulting NN [19]. Furthermore, the product of the spectral norms of the weights provides a trivial bound on an NN's Lipschitz constant and consequently, L1 and L2 regularization improve the robustness of an NN in the sense of Lipschitz continuity [4]. In this letter, we penalize a more accurate estimate of the Lipschitz constant, leading to a more direct and potentially more effective approach with the additional benefit of providing guaranteed Lipschitz bounds.

B. Lipschitz Constant Estimation

In the following, we outline a method to estimate bounds on the Lipschitz constant of multi-layer NNs exploiting the slope-restricted structure of the nonlinear activation functions, as it was shown in [13]. This method named LipSDP yields more accurate bounds than trivial bounds, i.e., the product of the spectral norms of the weights.

Continuous nonlinear activation functions $\varphi : \mathbb{R} \to \mathbb{R}$ can be interpreted as gradients of continuously differentiable, convex potential functions and consequently, they are slope-restricted, i.e., their slope is at least α and at most β,

$$\alpha \leq \frac{\varphi(x) - \varphi(y)}{x - y} \leq \beta \qquad \forall x, y \in \mathbb{R}, \tag{2}$$

where $0 \leq \alpha < \beta < \infty$. Consider the vector of activation functions $\phi_i : \mathbb{R}^{n_i} \to \mathbb{R}^{n_i}, \phi_i(z^i) = [\varphi(z_1^i) \cdots \varphi(z_{n_i}^i)]^\top$ and a fully-connected feed-forward NN with l hidden layers $f : \mathbb{R}^{n_0} \to \mathbb{R}^{n_{l+1}}$ described by the following equations

$$\begin{aligned} x^0 &= x, \\ x^{i+1} &= \phi_{i+1}(W^i x^i + b^i), \quad i = 0, \ldots, l-1, \\ f(x) &= W^l x^l + b^l, \end{aligned} \tag{3}$$

where $W^i \in \mathbb{R}^{n_{i+1} \times n_i}$ are the weight matrices and $b^i \in \mathbb{R}^{n_{i+1}}$ are the biases of the i-th layers, $n_0, \ldots, n_{l+1}$ being the dimension of the input, the neurons in the hidden layers, and the output. For every neuron, the slope-restriction property (2) can be written as an incremental quadratic constraint. Weighting these quadratic constraints with $\lambda_{ii} \geq 0$, or respectively, all activations with a diagonal weighting matrix $T \in \mathcal{D}_n := \{T \in \mathbb{R}^{n \times n} \mid T = \sum_{i=1}^n \lambda_{ii} e_i e_i^\top, \lambda_{ii} \geq 0\}$, results in an incremental quadratic constraint for the stacked activations:

$$\begin{bmatrix} \tilde{x} - \tilde{y} \\ \phi(\tilde{x}) - \phi(\tilde{y}) \end{bmatrix}^\top \underbrace{\begin{bmatrix} -2\alpha\beta T & (\alpha+\beta)T \\ (\alpha+\beta)T & -2T \end{bmatrix}}_{:=F(T)} \begin{bmatrix} \tilde{x} - \tilde{y} \\ \phi(\tilde{x}) - \phi(\tilde{y}) \end{bmatrix} \geq 0, \tag{4}$$

for all $T \in \mathcal{D}_n$, $\tilde{x}, \tilde{y} \in \mathbb{R}^n$ with $n = \sum_{i=1}^l n_i$. Herein, the inputs to all neurons are stacked up in one vector $\tilde{x}, \tilde{y}$, respectively, and the activation function $\phi : \mathbb{R}^n \to \mathbb{R}^n$, $\phi(\tilde{x}) = [\phi_1(x^1)^\top \cdots \phi_l(x^l)^\top]^\top$ is then applied to the concatenated vector. We now formulate an SDP based on [13] that exploits (4) for the estimation of an upper bound on the Lipschitz constant of the map characterized by the underlying NN using

$$A = \begin{bmatrix} W^0 & \ldots & 0 & 0 \\ \vdots & \ddots & \vdots & \vdots \\ 0 & \ldots & W^{l-1} & 0 \end{bmatrix}, \; B = \begin{bmatrix} 0 & I_n \end{bmatrix}.$$

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on September 29,2025 at 07:58:14 UTC from IEEE Xplore. Restrictions apply.

Theorem 1: Suppose there exist $L^2 > 0$, $T \in \mathcal{D}_n$ such that

$$\underbrace{\begin{bmatrix} A \\ B \end{bmatrix}^\top F(T) \begin{bmatrix} A \\ B \end{bmatrix} + \begin{bmatrix} -L^2 I & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & {W^l}^\top W^l \end{bmatrix}}_{=:P_l(L^2,T)} \preceq 0. \tag{5}$$

Then, (3) is globally Lipschitz continuous with Lipschitz bound $L \geq L^*$.

The proof directly follows from [20, A.4], the extended version of [13]. Note that this result differs from [13, Th. 2] as T in our case is a diagonal matrix. As we show in Section II-C, the result does not hold in general for the larger parametrization of T provided in [13]. The smallest value for the Lipschitz upper bound is determined by solving the SDP

$$\min_{L^2,T} L^2 \text{ s.t. } \quad P_l(L^2, T) \preceq 0, \;\; T \in \mathcal{D}_n, \tag{6}$$

where T and L^2 serve as decision variables.

Remark 1: In the case of one hidden layer the matrix P_1 reduces to

$$P_1(L^2, T) = \begin{bmatrix} -2\alpha\beta {W^0}^\top T W^0 - L^2 I & (\alpha+\beta){W^0}^\top T \\ (\alpha+\beta) T W^0 & -2T + {W^1}^\top W^1 \end{bmatrix}.$$

C. Counterexample for LipSDP With Coupling

In [13], three versions of the method LipSDP for Lipschitz constant estimation are stated that reconcile accuracy of the Lipschitz bound and computational complexity of the method by adjusting the number of the decision variables in the SDP. In this section, we give an illustrative counterexample to show that in [13, Ths. 1 and 2] are incorrect for the most accurate variant of LipSDP.

Reference [13, Th. 2] resembles Theorem 1 of this letter with the difference that in [13] a set of symmetric coupling matrices

$$\mathcal{T}_n = \{T \in \mathbb{R}^{n\times n} \mid T = \sum_{i=1}^{n} \lambda_{ii} e_i e_i^\top + \sum_{1\leq i<j\leq n} \lambda_{ij}(e_i - e_j)(e_i - e_j)^\top, \lambda_{ij} \geq 0\},$$

is introduced whereas we state Theorem 1 for a set of diagonal matrices $\mathcal{D}_n$. In the following, we give a minimal counterexample to show that, as suggested in this letter, a further restriction of the class of T is required. For that purpose, consider an NN with one hidden layer of size $n_1 = 2$, input and output size $n_0 = n_2 = 1$, activation function tanh and weights and biases

$$W^0 = \begin{bmatrix} -1 \\ -1 \end{bmatrix}, \; b^0 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, \; W^1 = \begin{bmatrix} -1 & 1 \end{bmatrix}, \; b^1 = -0.5.$$

The resulting NN approximates the cosine function on $x \in [-\frac{\pi}{2}, \frac{\pi}{2}]$ with a maximum error of 0.0843. Therefore, the maximum slope of the cosine gives a good approximation of the Lipschitz constant of this NN, which is ± 1 at $x = \pm\frac{\pi}{2}$, such that $L^* \approx 1$. However, the linear matrix inequality (LMI) (5) is feasible for arbitrarily small L^2 and $T = (e_1 - e_2)(e_1 - e_2)^\top \in \mathcal{T}_n$. An arbitrarily small L is obviously no upper bound on the Lipschitz constant of a cosine like function, thus contradicting [13, Th. 1].

To understand why in this case the LipSDP method fails to provide an upper bound on the Lipschitz constant, we look into the coupling of the neurons accounted for by $T \in \mathcal{T}_n$. Reference [13, Ths. 1 and 2] build on the assumption that (4) holds for all $T \in \mathcal{T}_n$ [13, Lemma 1]. In the following, we show by counterexample that [13, Lemma 1] is incorrect, i.e., that for a given slope-restricted function φ there are $\tilde{x}, \tilde{y} \in \mathbb{R}^n$ and $T \in \mathcal{T}_n$ that violate (4). We choose $n_1 = 2$, φ as the *ReLU* function that is slope-restricted in the sector [0, 1], and $T = (e_1 - e_2)(e_1 - e_2)^\top \in \mathcal{T}_n$. Let $x = \begin{bmatrix} 0 & 1 \end{bmatrix}^\top$ and $y = \begin{bmatrix} -1.5 & 0 \end{bmatrix}^\top$. Then evaluating (4) yields

$$\begin{bmatrix} \tilde{x} - \tilde{y} \\ \phi(\tilde{x}) - \phi(\tilde{y}) \end{bmatrix}^\top \begin{bmatrix} 0 & T \\ T & -2T \end{bmatrix} \begin{bmatrix} \tilde{x} - \tilde{y} \\ \phi(\tilde{x}) - \phi(\tilde{y}) \end{bmatrix} = -2 < 0,$$

which disproves [13, Lemma 1].

III. Training Robust NNs

In Section II-B, we stated a method that provides certificates on an NN's Lipschitz constant. In this section, we employ these certificates to design a training procedure for robust NNs. The proposed approach allows us to directly regularize the Lipschitz constant during training, which is only possible indirectly in regularization methods such as L2 regularization. We present two versions of it, the first one allows for minimization of the upper bound on the Lipschitz constant and the second one allows to enforce a desired bound on the Lipschitz constant.

A. Weights as Decision Variables

Eq. (6) can be used to assess an NN's robustness after training, whereas in this letter, to promote robustness during training, we use Eq. (6) to update the weights while minimizing the bound on the Lipschitz constant. Applying the Schur complement to (5) for $\alpha = 0$, the LMI can be rearranged, yielding an equivalent LMI that is linear in L^2 and $W = (W^0, \ldots, W^l)$, for fixed $T \in \mathcal{D}_n$:

$$M_l(L^2, W) := \begin{bmatrix} A \\ B \end{bmatrix}^\top \begin{bmatrix} 0 & \beta T \\ \beta T & -2T \end{bmatrix} \begin{bmatrix} A \\ B \end{bmatrix} + \begin{bmatrix} -L^2 I & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & {W^l}^\top \\ 0 & 0 & W^l & -I \end{bmatrix} \preceq 0. \tag{7}$$

Remark 2: For $\alpha > 0$, the underlying constraint is not convex in W. Consequently, we cannot state LMI constraints for $\alpha > 0$ and instead set $\alpha = 0$. This is a conservative choice for some activation functions, yet the tight lower bound for the most common ones., e.g., for ReLU and tanh the tight bounds are $\alpha = 0$ and $\beta = 1$ and the sigmoid function is slope-restricted with $\alpha = 0$ and $\beta = \frac{1}{4}$.

Remark 3: For the single-layer case, M_1 conveniently reduces to

$$M_1(L^2, W) = \begin{bmatrix} -L^2 I & \beta {W^0}^\top T & 0 \\ \beta T W^0 & -2T & {W^1}^\top \\ 0 & W^1 & -I \end{bmatrix}.$$

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on September 29,2025 at 07:58:14 UTC from IEEE Xplore. Restrictions apply.

While the Lipschitz constant estimation scheme in [13] optimizes over T, throughout the letter, we choose T to be a fixed matrix. This introduces conservatism into the framework and necessitates a suitable choice for T in order to keep the introduced conservatism to a minimum. For instance, the matrix T may be determined from the Lipschitz constant estimation outlined in Section II-B on the vanilla NN or the L2 regularized NN trained on the same problem.

B. Lipschitz Regularization

In general, NNs are trained on input-output data with the objective of minimizing a predefined loss, e.g., the mean squared error, cross-entropy, or hinge loss. We propose to not only minimize the NN's loss but also its Lipschitz constant. This yields an optimization problem with two separate objectives that can be solved conveniently using ADMM.

ADMM is an algorithm that solves optimization problems by splitting them into smaller subproblems that are easier to handle individually [21]. In order to apply the ADMM algorithm, the objective must be separable. The resulting subobjectives are then defined on uncoupled convex sets and are subject to linear equality constraints. The ADMM scheme solves the resulting optimization problem through independent minimization steps on the augmented Lagrangian of the optimization problem and a dual update step. The objectives at hand, i.e., the NN's loss and the Lipschitz bound, are indeed separable and defined on uncoupled convex sets. However, the problems are not completely independent and need to be connected through a linear constraint that requires the introduction of additional variables $\bar{W} = (\bar{W}^0, \dots, \bar{W}^l)$ of equal size as W. The loss of the NN $\mathcal{L}(W)$ is an explicit function of the weights W and the Lipschitz bound L depends on $\bar{W}$ through the LMI (7), yielding the following optimization problem:

$$\begin{aligned} \min_{W,\bar{W},L^2} \quad & \mathcal{L}(W) + \mu L^2 + \mathbf{1}_{M_l}(L^2, \bar{W}) \\ \text{s.t.} \quad & W = \bar{W} \end{aligned} \tag{8}$$

where $\mu > 0$ is a weighting parameter adjusting the trade-off between accuracy and robustness and

$$\mathbf{1}_M(L^2, \bar{W}) = \begin{cases} 0 & \text{if } M_l(L^2, \bar{W}) \preceq 0 \\ \infty & \text{if } M_l(L^2, \bar{W}) \succ 0 \end{cases}$$

is the indicator function. Applying the ADMM scheme to problem (8), results in the augmented Lagrangian function

$$\begin{aligned} \mathcal{L}_\rho(W, \bar{W}, L^2, Y) := \mathcal{L}(W) + \mu L^2 + \mathbf{1}_{M_l}(L^2, \bar{W}) \\ + \operatorname{tr}(Y(W - \bar{W})) + \frac{\rho}{2}\left\|W - \bar{W}\right\|^2 \end{aligned}$$

with Lagrange multipliers $Y^i \in \mathbb{R}^{n_i \times n_{i+1}}$, $Y = (Y^0, Y^1)$ and the penalty parameter $\rho > 0$. The optimum for (8) is then determined via the following iterative ADMM update steps:

$$W_{k+1} = \arg\min_{W} \mathcal{L}_\rho(W, \bar{W}_k, L_k^2, Y_k) \tag{9a}$$

$$(L_{k+1}^2, \bar{W}_{k+1}) = \arg\min_{L^2, \bar{W}} \mathcal{L}_\rho(W_{k+1}, \bar{W}, L^2, Y_k) \tag{9b}$$

$$Y_{k+1} = Y_k + \rho(W_{k+1} - \bar{W}_{k+1}). \tag{9c}$$

For training of robust NNs, we carry out the corresponding updates consecutively until convergence. The loss function is optimized analytically using backpropagation (Eq. (9a)) whereas the Lipschitz update step (9b) is an SDP, as implementation of the indicator function corresponds to an LMI constraint. Hence, the Lipschitz update step requires to solve an SDP in every iteration and thereby adds additional computations compared to the training of a vanilla NN.

Remark 4: It is possible to extend the framework and optimize over L^2, T, and W simultaneously which requires a second LMI constraint in (8) and an additional update step in (9), resulting in a multi-block ADMM scheme. This reduces conservatism but increases computation time.

C. Guaranteed Lipschitz Bounds

In Section III-B, we suggested to minimize an upper bound on the Lipschitz constant of an NN. Using the proposed ADMM framework, it is also possible to enforce a desired upper bound on the Lipschitz constant during training, yielding an NN with a guaranteed upper bound on the Lipschitz constant. In that case, the Lipschitz bound is not minimized but instead set to a desired value L_{des}. Judiciously, L^2 does not appear in the optimization objective of this setup

$$\begin{aligned} \min_{W,\bar{W}} \quad & \mathcal{L}(W) + \mathbf{1}_{M_l}(L_{\text{des}}^2, \bar{W}) \\ \text{s.t.} \quad & W = \bar{W} \end{aligned} \tag{10}$$

where W and $\bar{W}$ serve as decision variables. For enforcement of Lipschitz bounds, we apply the ADMM algorithm as in (9) to (10) instead of (8), i.e., we set $L = L_{\text{des}}$ instead of optimizing over L^2.

Theorem 2: When training a fully-connected NN with l hidden layers (3) by executing the ADMM scheme (9) for (10), L_{des} is an upper bound on the Lipschitz constant for the NN with weights $\bar{W}$ at all times during training.

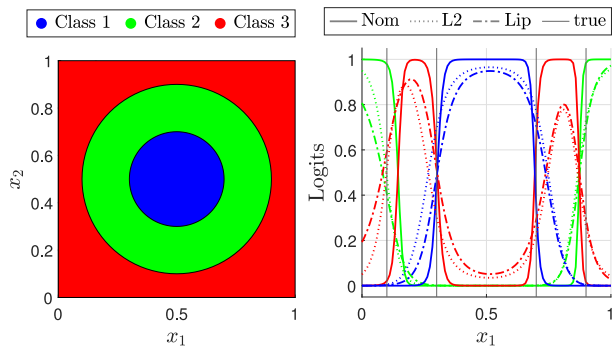
Proof: Theorem 2 follows from the fact that, by design, the bound L_{des} on the Lipschitz constant is enforced in every iteration of the ADMM scheme, more specifically, in every Lipschitz update step (9b) by the LMI constraint $M_l(L_{\text{des}}^2, \bar{W}) \preceq 0$ on the weights $\bar{W}$. ■

The training procedure based on (10) allows to choose the value of the Lipschitz bound and to train NNs with Lipschitz guarantees. This way, a desired degree of robustness can be directly enforced. However, the choice of such a constraint on L is always connected to the trade-off between accuracy and robustness, as the fit generally deteriorates when decreasing the Lipschitz constant constraint. In addition, it is helpful to initialize the weight parameters appropriately which does not only accelerate training but may also facilitate a better fit.

D. Convergence

ADMM was first introduced for optimization problems with convex subobjectives [21] and later, analyses of the ADMM scheme for non-convex objectives including further structural assumptions on the objective were formulated [22]. In the proposed framework, the loss $\mathcal{L}(W)$ clearly is not convex and has no obvious structural properties which renders a thorough convergence analysis complicated and beyond the scope of this letter. However, looking at the subproblems (9a) and (9b) separately, we point out that for (9a) gradient descent almost surely converges to local minima even for non-convex problems [23], and that (9b) is a semidefinite program with a

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on September 29,2025 at 07:58:14 UTC from IEEE Xplore. Restrictions apply.



(a) 2D data. (b) Logits at $x_2 = 0.5$.

Fig. 1. 2D classification example.

unique minimizer. Thus adding the convex regularization term and the indicator function of a convex set to the optimization problem of NN training, that converges reliably, does not add complexity in the form of non-convexity to the optimization problem.

E. Computational Tractability

Computational tractability and scalability of the proposed framework depend on the number of decision variables of the SDP. Generally, the complexity of SDP solvers scales cubically with the number of decision variables. Hence, for high-dimensional decision variables, solving an SDP is computationally more expensive than solving an unconstrained optimization problem using gradient descent. Therefore, the Lipschitz update step (9b) becomes the bottleneck of the proposed method as the number of neurons per hidden layers and the number of hidden layers, that together determine the size of the weights, increases. For example for picture inputs as commonly used in classification problems, the input dimension is usually high, potentially leading to high computation times or computational intractability. Nevertheless, incorporating convolutional or pooling layers, that can be formulated as constant feed-forward layers, in the formulation of M_l and P_l provides an option to improve computation time on larger scale problems and makes the proposed method indeed a worthwhile one to infer robustness to a neural network and obtain guarantees on the Lipschitz bound, also on large-scale problems. In Section IV, we show in an example that our method is applicable to MNIST, a typical benchmark classification problem. In order to keep the number of Lipschitz update steps to a minimum, we advise to first fully train a neural network without regularizers or with standard regularizers, such as the L2 regularizer, which serves as an initialization of the matrix T and the weights $(W, \bar{W})$.

IV. Simulation Results

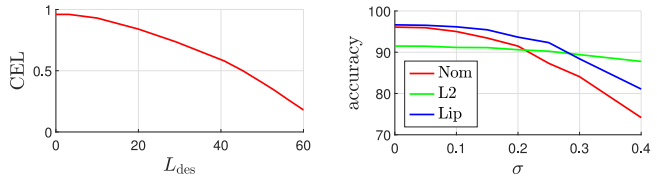
In this section, we illustrate the benefits of the presented framework for training of robust NNs on a 2D toy example and on MNIST. Our first illustrative example is a classification problem of 2D data with three classes, shown in Fig. 1(a). We design a feed-forward NN with two hidden layers of $n_1 = n_2 = 10$ neurons each, activation function tanh that is slope-restricted with $\alpha = 0$, $\beta = 1$, and the cross-entropy loss (CEL) as the loss function. For comparison, we train three NNs on 1000 training data points, a vanilla NN, an NN with L2 regularization (L2-NN) for benchmarking, see Section II-A, and finally the Lipschitz regularized NN (Lipschitz-NN) according to Section III-B, wherein the NN loss update step (9a) is solved using stochastic gradient descent and the SDP (9b) is solved using numerical SDP solvers [24], [25]. Before training, we initialize the Lipschitz-NN with the L2-NN with penalty parameter $\lambda = 3 \times 10^{-3}$. The hyperparameters $\rho = 0.25$ and $\mu = 1 \times 10^{-5}$ are chosen such that for comparability the Lipschitz constants of the L2-NN and the Lipschitz-NN are roughly the same. The CEL on training data, the accuracy on 50 test data points, the corresponding bound on the Lipschitz constant from (6) and the training time are computed for NNs with ten different weight initializations. The average results that are summarized in Table I show that Lipschitz regularization achieves a good accuracy while substantially reducing the Lipschitz bound compared to the nominal NN. Comparing the two regularizers, we observe that Lipschitz regularization leads to both a lower Lipschitz bound L, hence higher robustness, and a higher accuracy than L2 regularization. Note that regularizing the NN increases the cross-entropy between the ideal, one-hot encoded probability distribution and the predicted probability distribution of the NN which shows in the higher CEL values. Fig. 1(b) shows a projection at $x_2 = 0.5$ of the logits resulting from the three NNs for all three classes (blue, green, and red) onto the x_1-dimension. We clearly see the effect of a lower Lipschitz constant in the less steep slopes of the curves while the decision boundaries of the Lipschitz-NN remain accurate. The price to pay for the improved robustness is an increase in computation time (compare Table I), where the resulting training times suggest to explore alternative optimization algorithms to the iterative ADMM scheme, e.g., log-det barrier functions, that may potentially be more efficient. Note that for low-dimensional problems the Lipschitz update step (9b) is faster than the loss update step (9a), yet it becomes computationally expensive for high-dimensional problems (see Section III-E). In Fig. 2(a) we plot the CEL of NNs trained using (10) for different Lipschitz bounds L_{des} to visualize the trade-off between accuracy and robustness.

TABLE I
CEL, Accuracy, Lipschitz Bounds, Training Times for 2D and MNIST Experiment

	CEL	Accuracy	L	L_{triv}	$\bar{t}_{\text{tr}}$ (loss/Lip step)
Nom-NN	0.15	96.0%	339.1	710.5	19,0s
L2-NN	0.42	80.5%	56.7	109.5	28.4s
Lip-NN	0.37	88.7%	49.4	134.2	164.9s (41.3s/0.63s)
Nom-NN	0.10	96.5%	140.6	879.1	202.8s
L2-NN	0.27	90.4%	15.3	23.2	225.5s
Lip-NN	0.21	96.3%	13.2	24.0	4632s (854s/521s)

In our second example, we apply the Lipschitz regularization framework to the benchmark data set MNIST of input dimension 28×28 [26]. We construct a feed-forward NN with one pooling layer, two hidden layers with $n_1 = n_2 = 50$ neurons, the activation function tanh and the CEL. Again, we train three NNs, a vanilla NN, an L2-NN with penalty parameter $\lambda = 3 \times 10^{-3}$, and a Lipschitz-NN with $\rho = 0.25$ and $\mu = 0.01$, which is initialized from the L2-NN. Note that the pooling layer is formulated as a constant feed-forward layer to be included as W^0 into LMI (7), where W^0 is not a decision

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on September 29,2025 at 07:58:14 UTC from IEEE Xplore. Restrictions apply.



(a) Trade-off between accuracy and robustness on 2D-example.

(b) Accuracy on noise corrupted MNIST test data.

Fig. 2. Robustness experiments.

variable. The method also works without a pooling layer, yet the Lipschitz step becomes significantly more time-consuming. From the resulting accuracies and bounds on the Lipschitz constant shown in Table I, we conclude that on MNIST our framework also finds an NN with a low Lipschitz constant but comparable accuracy to the nominal NN, while the accuracy of an L2-NN with a comparably low Lipschitz constant is significantly compromised. Fig. 2(b) shows the evaluation of the three NNs on noise corrupted data, that was created by adding Gaussian noise $\mathcal{N}(0, \sigma^2)$ to the (0, 1)-normalized MNIST data and averaging over ten different noise samples. Advantages of the Lipschitz-NN become apparent for Gaussian noise with low standard deviation.

Altogether, the results show that Lipschitz regularization can be used to effectively train robust NNs while trading off robustness and accuracy. Code to reproduce the examples can be found at https://github.com/st157640/Training-robust-neural-networks-using-Lipschitz-bounds.git.

V. Conclusion

We proposed a framework for training of multi-layer NNs that encourages robustness, by both considering Lipschitz regularization and by enforcing Lipschitz bounds during training. The underlying SDP [13] estimates the upper bound on the Lipschitz constant more accurately than traditional methods as it exploits the fact that activation functions are slope-restricted. We designed an optimization scheme based on this SDP that trains an NN to fit input-output data and at the same time increases its robustness in terms of Lipschitz continuity. We used ADMM to solve the underlying optimization problem and to therein conveniently incorporate the trade-off between accuracy and robustness. In addition, we presented a variation of the framework that allows to bound the Lipschitz constant by a desired value, i.e., training NNs with robustness guarantees. We successfully tested our method on two examples where we benchmarked it with L2 regularization.

Next steps include the application of our method to control problems by using LMI constraints to verify and enforce properties, such as closed-loop stability, on feedback interconnections that include NN controllers. Also, we plan to explore alternatives to the ADMM algorithm that solve the underlying optimization problem in an accelerated manner. In addition, for benchmarking purposes, we plan to compare the proposed methods to other training procedures that improve robustness, including methods based on other - not necessarily SDP based - Lipschitz estimation schemes.

References

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Proc. NeurIPS*, Lake Tahoe, NV, USA, 2012, pp. 1097–1105.

[2] R. Collobert, J. Weston, L. Bottou, M. Karlen, K. Kavukcuoglu, and P. Kuksa, "Natural language processing (almost) from scratch," *J. Mach. Learn. Res.*, vol. 12, no. 76, pp. 2493–2537, 2011.

[3] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[4] C. Szegedy *et al.*, "Intriguing properties of neural networks," 2013. [Online]. Available: arXiv:1312.6199.

[5] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2014. [Online]. Available: arXiv:1412.6572.

[6] N. Papernot, P. McDaniel, X. Wu, S. Jha, and A. Swami, "Distillation as a defense to adversarial perturbations against deep neural networks," in *Proc. Symp. SP*, San Jose, CA, USA, 2016, pp. 582–597.

[7] E. Wong and Z. Kolter, "Provable defenses against adversarial examples via the convex outer adversarial polytope," in *Proc. ICML*, Stockholm, Sweden, 2018, pp. 5283–5292.

[8] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," 2017. [Online]. Available: arXiv:1706.06083.

[9] Y. Tsuzuku, I. Sato, and M. Sugiyama, "Lipschitz-margin training: Scalable certification of perturbation invariance for deep neural networks," in *Proc. NeurIPS*, Montreal, QC, Canada, 2018, pp. 6542–6551.

[10] P. Combettes and J.-C. Pesquet, "Lipschitz certificates for layered network structures driven by averaged activation operators," 2020. [Online]. Available: arXiv:1903.01014.

[11] A. Virmaux and K. Scaman, "Lipschitz regularity of deep neural networks: analysis and efficient estimation," in *Proc. NeuRIPS*, Montreal, QC, Canada, 2018, pp. 3839–3848.

[12] F. Latorre, P. Rolland, and V. Cevher, "Lipschitz constant estimation of neural networks via sparse polynomial optimization," 2020. [Online]. Available: arXiv:2004.08688.

[13] M. Fazlyab, A. Robey, H. Hassani, M. Morari, and G. Pappas, "Efficient and accurate estimation of Lipschitz constants for deep neural networks," in *Proc. NeurIPS*, Vancouver, BC, Canada, 2019, pp. 11423–11434.

[14] T.-W. Weng *et al.*, "Towards fast computation of certified robustness for ReLU networks," 2018. [Online]. Available: arXiv:1804.09699.

[15] M. Jordan and A. G. Dimakis, "Exactly computing the local Lipschitz constant of ReLU networks," 2020. [Online]. Available: arXiv:2003.01219.

[16] M. Cisse, P. Bojanowski, E. Grave, Y. Dauphin, and N. Usunier, "Parseval networks: Improving robustness to adversarial examples," in *Proc. ICML*, Sydney, NSW, Australia, 2017, pp. 854–863.

[17] H. Gouk, E. Frank, B. Pfahringer, and M. Cree, "Regularisation of neural networks by enforcing Lipschitz continuity," 2018. [Online]. Available: arXiv:1804.04368.

[18] M. Hein and M. Andriushchenko, "Formal guarantees on the robustness of a classifier against adversarial manipulation," in *Proc. NeurIPS*, Long Beach, CA, USA, 2017, pp. 2266–2276.

[19] A. Krogh and J. A. Hertz, "A simple weight decay can improve generalization," in *Proc. NeurIPS*, Denver, CO, USA, 1991, pp. 950–957.

[20] M. Fazlyab, A. Robey, H. Hassani, M. Morari, and G. J. Pappas, "Efficient and accurate estimation of Lipschitz constants for deep neural networks," 2019. [Online]. Available: arXiv:1906.04893.

[21] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Found. Trends® Mach. Learn.*, vol. 3, no. 1, pp. 1–122, 2011.

[22] Y. Wang, W. Yin, and J. Zeng, "Global convergence of ADMM in nonconvex nonsmooth optimization," *J. Sci. Comput.*, vol. 78, no. 1, pp. 29–63, 2019.

[23] J. D. Lee, M. Simchowitz, M. I. Jordan, and B. Recht, "Gradient descent only converges to minimizers," in *Proc. COLT*, New York, NY, USA, 2016, pp. 1246–1257.

[24] J. Löfberg, "Yalmip: A toolbox for modeling and optimization in MATLAB," in *2004 IEEE int. conf. on robotics and automation*, Taipei, Taiwan, pp. 284–289. Sep. 2004.

[25] MOSEK ApS. (2019). *The MOSEK Optimization Toolbox for MATLAB Manual. Version 9.0.* [Online]. Available: http://docs.mosek.com/9.0/toolbox/index.html

[26] Y. LeCun. *The MNIST Database of Handwritten Digits*. Accessed: Apr. 1, 2020. [Online]. Available: http://yann.lecun.com/exdb/mnist/

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on September 29,2025 at 07:58:14 UTC from IEEE Xplore. Restrictions apply.